

BirdCLEF+ 2026

Kaggle Competition — Technical Report

Notebook	ahmed-ali.ipynb (Competition Notebook, Version 1 of 1)
Competition	BirdCLEF+ 2026 (Kaggle, hosted competition)
Task	Multi-label bioacoustic species classification
Execution mode	MODE = "submit" (inference / submission-generation run)
Environment	Kaggle Notebook — CPU only, no internet access
Language / runtime	Python 3.12
Notebook runtime (Kaggle-reported)	7m 5s
Private Score	0.93842
Best Score	0.93842 (Version 1)

Final Kaggle Result: Private Leaderboard Score = **0.93842** (macro ROC-AUC) — Best Score achieved, Version 1.

1. Competition Overview

BirdCLEF 2026 is a Kaggle-hosted bioacoustics competition in which participants build models that detect and classify bird (and other animal) vocalizations inside long-form audio "soundscape" recordings. Each training/test recording is a 60-second .ogg clip, split into twelve consecutive 5-second windows. For every window the model must output a probability for every species in the competition's taxonomy, and submissions are scored by a macro-averaged ROC-AUC across species.

The dataset used by this notebook (from the metadata attached to the Kaggle kernel) includes a taxonomy file, a sample submission template, train soundscapes with weak/strong labels, hidden test soundscapes, and a pretrained Google "Perch" bird-vocalization embedding/classifier model, along with several externally-provided datasets (ONNX runtime wheel, a converted Perch ONNX model, and a distilled Sound Event Detection (SED) model).

As inspected from the notebook: 234 target species classes, 59 fully-labeled 60-second training soundscapes yielding 708 labeled 5-second windows, of which 71 classes had at least one positive label in that fully-labeled subset.

Kaggle Notebook Inputs

Category	Item
Competition	BirdCLEF+ 2026
Dataset	BC2026-Distilled-SED-Public — distilled Sound Event Detection ONNX model
Dataset	perch_meta — precomputed Perch embeddings/scores metadata cache
Dataset	Perch-onnx-for-birdclef+2026 — ONNX conversion of Perch v2 + onnxruntime wheel
Model	Google Perch — bird-vocalization-classifier (TensorFlow2, V1)
Notebook	tf_wheels — local TensorFlow/TensorBoard 2.20 install wheels

2. Problem-Solving Approach / Pipeline

The notebook does not train a model from scratch. Instead it builds a multi-stage ensemble system on top of a pretrained embedding model, combining classical machine learning, a custom sequence model, domain priors, and post-processing heuristics. The pipeline proceeds through the following stages:

- **Environment & reproducibility setup.** Installs an ONNX Runtime wheel and TensorFlow 2.20 from local Kaggle dataset wheels (no internet access), and fixes a global random seed (42) across Python, NumPy and PyTorch for reproducibility.
- **Mode & configuration.** A single MODE flag ("train" vs "submit") drives a central CFG dictionary that scales epoch counts, patience, and cross-validation folds — fewer epochs/folds in submit mode to fit the inference time budget.
- **Data loading & label engineering.** Loads taxonomy, sample submission and train_soundscape_labels.csv; aggregates multi-label annotations per (file, time-window) into multi-hot label vectors; parses filename metadata (recording site, hour of day, UTC) via regex; isolates the subset of soundscapes that are fully labeled across all 12 windows for reliable supervision.
- **Perch embedding/classifier inference.** Loads Google's pretrained "Perch v2" bird-vocalization model (TensorFlow SavedModel, with an ONNX-converted version used when available for ~150x faster CPU inference). For every 5-second window it extracts both a 1536-dimensional embedding and raw per-species logits/scores.
- **Taxonomy mapping & genus-level proxies.** Maps the competition's 234 species to Perch's own label space by scientific name; species with no direct Perch output (31 of 234) are partially covered via genus-level proxy matching for amphibian/insect/bird taxa, recovering usable signal for 3 additional species.
- **Caching.** Perch scores and embeddings are cached to disk (parquet + npz) so repeated runs / experiments don't recompute the expensive forward pass; the code also checks for pre-built external caches.
- **Evaluation harness.** Defines the official competition metric (macro ROC-AUC, skipping classes with zero positive labels) and a GroupKFold-by-filename out-of-fold (OOF) evaluator so that windows from the same recording never leak across train/validation folds.
- **Post-processing / signal-shaping utilities.** A family of interchangeable, hand-tuned post-processing functions: temporal neighbor smoothing, file-level confidence scaling (top-k window boosting), rank-aware scaling by each file's peak confidence, and "adaptive delta" smoothing that smooths uncertain windows more than confident ones.

- **Site/hour prior blending.** Builds empirical priors of species occurrence by recording site, hour-of-day, and site×hour combinations from the training data, then blends these priors into the raw scores to account for known ecological/temporal patterns.
- **Per-class temperature scaling.** Applies a simple two-bucket temperature adjustment, treating "texture" taxa (amphibians, insects) differently from ordinary event-like bird calls.
- **MLP probe classifiers on embeddings.** Trains lightweight per-species MLP classifiers (scikit-learn) on PCA-reduced Perch embeddings (64 components, ~81% variance retained) for species with at least 5 positive example windows; for the observed run this trained 58 species-specific probes. These are later vectorized into a single batched PyTorch module for fast inference.
- **Calibration & thresholding.** Applies isotonic regression per species on OOF predictions to calibrate probability scores, then grid-searches an F1-optimal decision threshold per species (mean optimized threshold ≈ 0.469 across 31 calibrated classes in the run shown).
- **Custom sequence models (ProtoSSM / ResidualSSM).** Implements a custom Selective State-Space Model (an architecture in the Mamba/S4 family) plus a residual-correction SSM, combining per-window Perch embeddings/scores with site and hour metadata and prototype-based classification and cross-window attention across the 12 windows of a file. This model is trained per fold (or once in submit mode) and produces its own species probability outputs.
- **Independent SED branch.** A second, independently pretrained Sound Event Detection model (ONNX, distilled, mel-spectrogram input at 256 mels) is run over the same audio to produce an alternative set of per-window species probabilities.
- **Ensembling & final blending.** Combines the ProtoSSM branch and the SED branch using percentile-rank blending (60% ProtoSSM-derived, 40% SED-derived), with additional conditional rules that shift the blend weight when the two branches strongly agree or strongly disagree, a temporal smoothing kernel over neighboring windows within a file, "sonotype mirroring" for acoustically similar species groups, and adaptive thresholding for rare species — producing the final submission.csv.

3. Project Flow & Methodology

The diagram below summarizes the end-to-end pipeline as implemented in the notebook, from raw audio to the final blended submission file.



Reading the flow: the pipeline is best understood as two parallel "expert" branches — a learned sequence model (ProtoSSM, built on Perch embeddings + MLP-probe-refined scores) and an independent SED model — which are trained/run separately and only combined at the very last step via rank-based ensembling and rule-based conditional blending, rather than a single end-to-end trained network.

4. Model / Component Summary

Component	Type	Role
-----------	------	------

Perch v2	Pretrained CNN (Google), TF SavedModel + ONNX	Base embeddings (1536-d) + per-species logits
MLP probes	scikit-learn MLPClassifier, per species	Refines Perch embeddings into calibrated species scores
LightProtoSSM	Custom Selective State-Space Model (PyTorch)	Sequence model over the 12 windows/site/hour metadata
ResidualSSM	Custom SSM (PyTorch)	Residual correction layer on top of ProtoSSM
Distilled SED	ONNX CNN on mel-spectrograms	Independent second opinion for ensembling
Isotonic Regression	Classical calibration	Per-species probability calibration
Priors (site/hour)	Empirical frequency tables	Bayesian-style prior blending

5. Evaluation Metric & Accuracy / Score

Metric: macro-averaged ROC-AUC across species (classes with zero positive labels in the evaluation set are excluded), computed with scikit-learn's `roc_auc_score`. This matches the official BirdCLEF+ 2026 leaderboard metric.

Cross-validation methodology: the notebook defines an "honest" out-of-fold evaluator using `GroupKFold` grouped by filename (3 folds in submit mode, 5 in train mode), specifically to avoid the ~0.01–0.03 AUC inflation that would occur if windows from the same recording leaked across train/validation splits. In the captured execution, `MODE` was set to "submit" (`CFG["run_oof"]=False`), so this OOF cell was skipped for speed and the notebook instead ran a 20-file dry-run purely to validate that the inference/submission code executes correctly — the dry-run itself produces no score.

Official Kaggle result: the notebook was submitted to the BirdCLEF+ 2026 competition and scored on the hidden leaderboard as follows:

Leaderboard	Score (macro ROC-AUC)
Private Score	0.93842
Best Score (all versions)	0.93842 — Version 1

A macro-AUC of 0.93842 indicates strong overall species-discrimination performance across the 234-way multi-label taxonomy, averaged evenly across species regardless of how rare or common they are in the data — this is what makes the metric sensitive to performance on rare species, which the pipeline specifically targets via class-frequency weighting, genus-level proxy mapping, and adaptive rare-species thresholding.

Note: the notebook's own `honest_oof_auc()` function was not exercised in this particular run (submit mode), so no local/offline macro-AUC number is available from the notebook's own logs; the 0.93842 figure above is the official Kaggle-reported score for this submission, taken from the notebook's competition page.

6. Key Statistics Observed in This Run

Metric	Value
Target species (classes)	234
Fully-labeled training soundscapes	59
Labeled 5-second training windows	708
Species with ≥ 1 positive label in labeled subset	71 / 234
Species directly mapped to a Perch logit	203 / 234
Unmapped species recovered via genus proxy	3 (of 31 unmapped)
Perch embedding dimensionality	1536
Perch inference pass time (708 windows, cache build)	≈ 126.8 s
Embedding PCA reduction	1536 $\rightarrow$ 64 dims ($\approx 81.5\%$ variance retained)
Species-specific MLP probes trained	58 (species with ≥ 5 positive windows)

ProtoSSM (sequence model) training time, submit mode	≈12.0 s
Species calibrated with isotonic regression	31
Mean optimized decision threshold	≈0.469 (range 0.25–0.50)
Final ensemble blend (ProtoSSM : SED)	60% : 40% (rank-based, with conditional adjustments)
Dry-run inference set (no hidden test present)	20 training files (240 windows)

7. External Datasets & Models Used

According to the notebook's Kaggle metadata, this kernel attaches the following external resources (in addition to the official competition data):

- The official BirdCLEF 2026 competition dataset (taxonomy, train/test soundscapes, sample submission).
- Google's pretrained "bird-vocalization-classifier" (Perch v2) TensorFlow model, sourced from Kaggle Models.
- A community-provided ONNX conversion of Perch v2 plus a matching onnxruntime wheel, for faster CPU inference.
- A precomputed Perch embeddings/scores cache from another public Kaggle notebook/dataset (used opportunistically if present, to skip recomputation).
- A distilled Sound Event Detection (SED) ONNX model, used as an independent second model for the final ensemble.

8. Notable Engineering Choices

- Runs entirely offline (isGpuEnabled=False, isInternetEnabled=False) — all dependencies are installed from local wheel files bundled as Kaggle datasets, and CPU-only ONNX inference is preferred for ~150x speed-up over the TensorFlow SavedModel path.
- GroupKFold by filename is used everywhere cross-validation appears, to prevent leakage between windows of the same recording — a common pitfall in soundscape-labeled bioacoustic tasks.
- A single MODE switch ("train"/"submit") controls training epochs, patience, and whether OOF evaluation runs, letting the same notebook serve both experimentation and the time-boxed Kaggle submission run.
- Class-imbalance handling appears throughout: frequency-based class weighting for the MLP probes, focal-loss style parameters and label smoothing configured for the ProtoSSM, and a minimum-positive-count cutoff before training a per-species probe.
- The final prediction is an ensemble of a learned sequence model (ProtoSSM/ResidualSSM) and an independent SED model, combined via percentile-rank blending rather than raw-probability averaging, with conditional rules ("fake_only", "proto_cont", "sed_only") to decide which branch to trust per-row based on how strongly each model agrees with itself and the other branch.
- Post-processing includes ecological/temporal priors (site and hour of day), per-class temperature scaling, isotonic calibration, F1-optimal thresholding, and multiple smoothing/scaling passes over neighboring time windows.

This report was generated automatically from the contents and saved cell outputs of ahmed-ali.ipynb. It reflects only what is present in the notebook (code, comments, and printed outputs); no leaderboard score is fabricated where the notebook itself does not contain one.